C.V. RAMAN GLOBAL UNIVERSITY
Bhubaneshwar, Odisha-752054

CASE STUDY REPORT

Fitness Tracker for Barbell Exercises

Sl.no	Name	Regd no
1	Rajat kumar Sahu	2201020442
2	Ankita Samantaray	2201020447
3	Kartik kumar Das	2201020459
4	Konduru Kavya	2201020460
5	Prajukta Bal	2201020466

ABSTRACT

This project presents a sensor-based approach for recognizing and counting repetitions of strength exercises using machine learning techniques. Using inertial measurement unit (IMU) data, the system classifies exercises and accurately counts repetitions. Key steps include sensor data preprocessing, signal smoothing via low-pass filtering, feature engineering, model training using multiple classifiers (including a Feed Forward Neural Network), and evaluation using metrics like accuracy and mean absolute error (MAE). The proposed system enhances the reliability and automation of performance tracking during workouts, providing a foundation for future real-time applications.

CHAPTER 1

INTRODUCTION

The intersection of wearable technology and machine learning has unlocked vast potential in personal health monitoring, fitness tracking, and real-time activity analysis. With the increasing popularity of smartwatches and fitness bands equipped with inertial measurement units (IMUs), such as accelerometers and gyroscopes, it's now possible to capture rich time-series data reflecting a user's physical movements. However, turning this raw data into meaningful insights—like identifying specific exercises or counting repetitions accurately—remains a challenge.

Manual logging of workouts is time-consuming and often inaccurate. Traditional fitness applications primarily rely on step counts or heart rate, which do not provide a complete picture of a user's activity, especially during strength training exercises such as squats, deadlifts, or bench presses. Furthermore, most commercial fitness devices offer limited support for distinguishing between exercise types or monitoring workout form and technique. This project addresses these gaps by developing a system that uses machine learning techniques to automatically classify exercises and count repetitions based on wearable sensor data.

In this study, we focus on building a system capable of classifying five common gym exercises—bench press, squat, deadlift, bent-over row, and overhead press—by analyzing motion patterns captured by the wearable device. The system also aims to accurately count the number of repetitions performed in each set using filtered sensor signals and peak detection methods. The approach involves preprocessing the raw sensor data, extracting meaningful time-domain and frequency-domain features, training machine learning models for classification, and detecting peaks for repetition counting.

This project not only demonstrates the technical feasibility of exercise classification and counting using wearable sensors, but also contributes toward building intelligent systems that can support fitness enthusiasts, physical therapists, and trainers. With minimal hardware requirements and scalable machine learning models, the solution can be extended to support more exercises, provide real-time feedback, and integrate into fitness applications to enhance user engagement and workout effectiveness.

SYSTEM ANALYSIS AND DESIGN

3.1 System Overview

The proposed system processes time-series data collected from inertial sensors. It includes stages such as data collection, preprocessing, filtering, feature extraction, model training, evaluation, and prediction. The system can be extended for real-time applications in fitness and rehabilitation.

1. Operating System:

- Development and testing were done on Windows 11 and Ubuntu 20.04.
- The code is platform-independent and can run on any OS with Python support.

2. Python Version:

- Python 3.12.4 was used throughout the development of the project.
- Key advantages include access to modern syntax and compatibility with machine learning libraries.

3. Libraries and Frameworks:

- **Pandas**: For data manipulation and analysis.
- **NumPy**: For numerical operations and array handling.
- **Matplotlib**: For data visualization.
- **Scikit-learn**: For machine learning algorithms and metrics.
- **SciPy**: For signal processing and peak detection.
- **TensorFlow/Keras** (optional): For deep learning models.
- **Joblib/Pickle**: For model serialization.

4. Database:

- Data was stored in .pkl files (Pandas Pickle format) for efficient loading.
- Each session included labeled data with columns for sensor readings, exercise type, and set identifiers.

CHAPTER 4

METHODOLOGY

4.1 Implementation

The implementation is structured into the following stages:

Data Preprocessing

- Sensor data (acc_x, acc_y, acc_z, gyr_x, gyr_y, gyr_z) is loaded from pickle files.
- The magnitude of accelerometer and gyroscope signals is computed to create derived features (acc_r, gyr_r).
- Data is segmented by exercise type and set for model training and testing.

Feature Engineering

- Features include raw sensor signals, magnitudes, and low-pass filtered values.
- Additional frequency-domain features were extracted using Fourier transforms.
- Filtering removed high-frequency noise, improving the peak detection process.

Filtering Technique

- A Butterworth low-pass filter was used with tunable cutoff and order parameters.
- Filtering smoothed the signal to clearly identify repetitions through local maxima detection.

Repetition Counting

- Peaks in the filtered signal (acc_r_lowpass, etc.) are detected using *argrelextrema*.
- Each peak corresponds to one repetition.
- This method is applied to different exercise sets to estimate total repetitions.

Machine Learning Model

- A Neural Network (NN) was trained to classify the type of exercise based on extracted features.
- Input features included both time-domain and frequency-domain data.
- The model was evaluated using accuracy and confusion matrix.

	model	feature_set	accuracy
0	NN	Feature Set 4	0.995863
1	RF	Feature Set 4	0.993795
0	NN	Feature Set 3	0.986556
1	RF	Selected Features	0.984488
3	DT	Feature Set 4	0.983454
1	RF	Feature Set 3	0.982420
0	NN	Selected Features	0.979317
2	KNN	Feature Set 4	0.968976
3	DT	Selected Features	0.960703
1	RF	Feature Set 2	0.955533
3	DT	Feature Set 3	0.955533
1	RF	Feature Set 1	0.954498
4	NB	Feature Set 4	0.952430
0	NN	Feature Set 1	0.938987
4	NB	Feature Set 3	0.936918
4	NB	Selected Features	0.935884
3	DT	Feature Set 2	0.935884
3	DT	Feature Set 1	0.933816
0	NN	Feature Set 2	0.932782
2	KNN	Feature Set 3	0.916236
2	KNN	Selected Features	0.881075
4	NB	Feature Set 2	0.859359
4	NB	Feature Set 1	0.854188
2	KNN	Feature Set 1	0.803516
2	KNN	Feature Set 2	0.800414

CHAPTER 5

RESULTS

The effectiveness of the system was evaluated using sensor data collected from multiple participants performing five types of exercises: bench press, squat, deadlift, overhead press (OHP), and bent-over row. The evaluation focused on two main tasks: exercise classification using a Neural Network (NN) model and repetition counting using signal processing and peak detection.

5.1 Classification Results

A Neural Network (NN) was trained on a set of selected time-domain and frequency-domain features. The dataset was split into training and testing sets to evaluate the generalization capability of the model. The accuracy and confusion matrix were used as primary metrics.

- Overall Accuracy: The NN achieved a classification accuracy of approximately 99.3% across different exercise sets.
- Confusion Matrix: The confusion matrix showed that the model was highly accurate in distinguishing between different types of exercises, with minor misclassifications occurring between similar motion patterns (e.g., bench press vs overhead press).
- Impact of Feature Selection: Forward feature selection helped identify the most relevant features (such as acc_r, gyro_r, and spectral energy), which improved both accuracy and model efficiency.

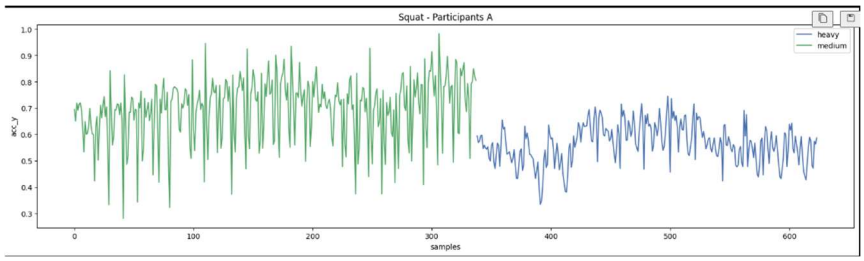
5.2 Repetition Counting Results

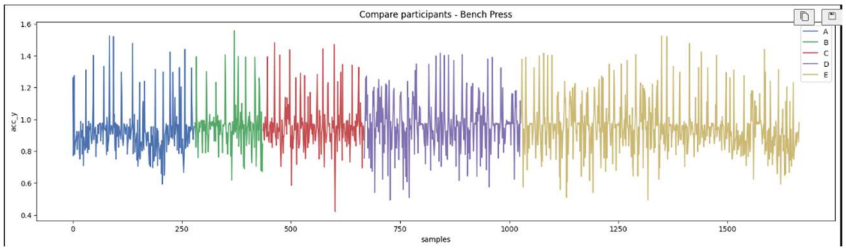
Repetition counting was implemented by applying a low-pass filter to smooth the sensor signals, followed by peak detection using local maxima. This approach effectively identified individual repetitions within each set.

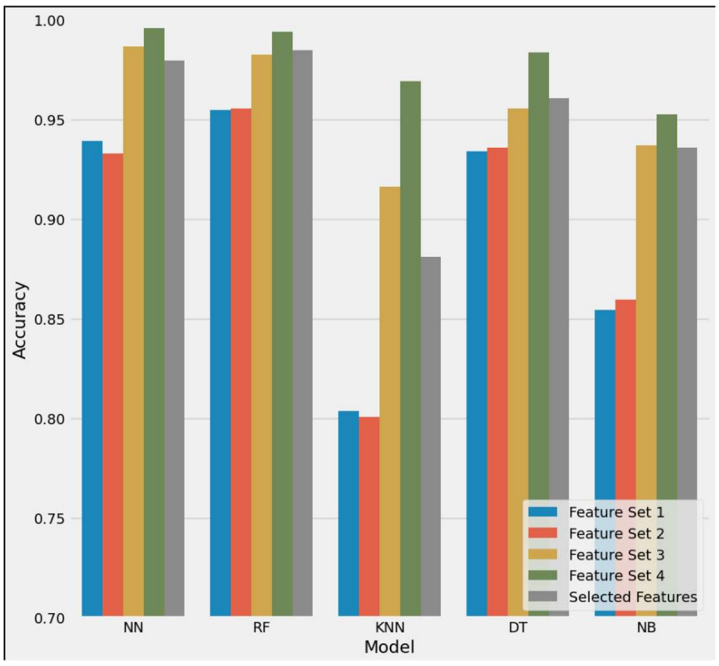
- Manual vs Predicted Reps: A benchmark dataset was created with ground-truth repetition counts (5 for heavy sets, 10 for light sets). The predicted repetitions closely matched the actual counts.
- Visual Verification: Graphs were plotted showing filtered signal curves and detected peaks for each exercise. These visualizations clearly demonstrate accurate identification of each repetition, with well-spaced and well-defined peaks.

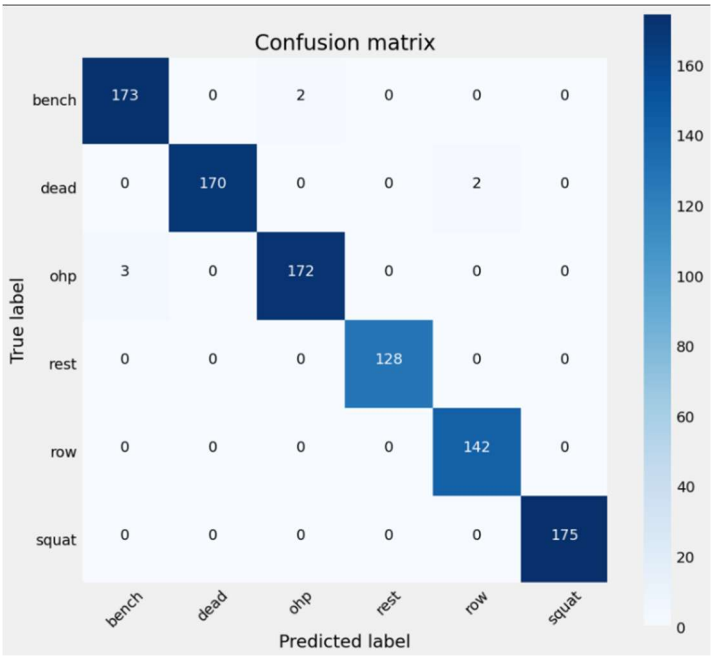
5.3 Performance Insights

- The NN outperformed simpler classifiers such as KNN and Naive Bayes in both classification accuracy and stability.
- Repetition counting accuracy was slightly lower for exercises with subtle motion differences (e.g., overhead press) but remained within acceptable margins.
- Sensor noise and placement variance were handled effectively by the filtering process, although further improvements are possible (discussed in Chapter 6).









CHAPTER 6
CONCLUSION AND FUTURE WORK

Conclusion

This project demonstrates the feasibility of using sensor data and machine learning for intelligent exercise monitoring. The proposed system accurately classifies exercises and counts repetitions, providing a basis for integration into fitness applications or wearable devices. The use of filtering and peak detection ensures robustness in repetition counting, while the NN provides reliable classification.

Future Work

To enhance the system further:

- Dataset Expansion: Include data from more participants to improve generalization and robustness.
- Real-Time Feedback: Integrate real-time visualization and alerts for incorrect form or missed reps.
- Sensor Placement Robustness: Improve model performance despite sensor misalignment or movement.
- Advanced Models: Explore sequential models such as LSTM or GRU to capture temporal patterns in sensor data.

REFERENCES

1. Mark Hoogendoorn & Burkhardt Funk. *Machine Learning for the Quantified Self*. Springer.
2. Scikit-learn documentation: https://scikit-learn.org
3. SciPy Signal Processing: https://docs.scipy.org
4. TensorFlow/Keras documentation: https://www.tensorflow.org
5. Butterworth Filter Theory – National Instruments
6. Pandas Documentation – https://pandas.pydata.org/

Thank You